BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EEE 304

Digital Electronics laboratory

Final Project Report

Section: A2 Group: 01

Group Elevator Controller

Course Instructors:
Full Name 1: Tanushri Medha Kundu
Full Name 2: Archishman Sarkar

Signature of Instructor: _____

Academic Honesty Statement

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: Alauddin. **Full Name:** *Alauddin Ahmed* **Student ID:** 2106034	Signature: Emon **Full Name:** *Rakibul Islam Emon* **Student ID:** *2106040*
Signature: Wassie **Full Name:** *Wassie Haque* **Student ID:** *2106041*	Signature: Sanjay **Full Name:** *Sanjay Sen* **Student ID:** *2106054*

Table of Contents

1.Abstract 4

2. Introduction 5

3. Design 6

3.1 Problem Formulation (PO(b)) 6

3.1.1 Identification of Scope 6

3.1.2 Literature Review…….. 7

3.1.3 Formulation of Problem 8

3.1.4 Analysis 9

3.2 Design Method (PO(a)) 14

3.3 Full Source Code of Firmware 16

4 Implementation 24

5 Design Analysis and Evaluation 25

5.1 Novelty 25

5.2 Design Considerations (PO(c)) 25

5.2.1 Considerations to public health and safety 25

5.2.2 Considerations to environment..... 26

5.2.3 Considerations to cultural and societal needs 26

5.3 Investigations (PO(d)) 27

5.3.1 Design of Experiment 27

5.3.2 Data Collection..... 28

5.3.3 Results and Analysis 28

5.3.4 Interpretation and Conclusions on Data 29

5.4 Limitations of Tools (PO(e)) 29

5.5 Impact Assessment (PO(f))..... 29

5.5.1 Assessment of Societal and Cultural Issues 29

5.5.2 Assessment of Health and Safety Issues.....29

5.5.3 Assessment of Legal Issues.....30

5.6 Sustainability Evaluation……..... 30

6. Reflection on Individual and Team work 31

6.1 Individual Contribution of Each Member..... 31

6.2 Log Book of Project Implementation 32

7. Communication to External Stakeholders……..33

7.1 GitHub Link…….……33

8. Future Work (PO(l)) 34

9. References 35

1.Abstract

This project presents the design and implementation of a **two-elevator, four-floor group control system using Verilog Hardware Description Language (HDL)**. The system is modeled using **finite state machines (FSMs)** for each elevator car, combined with a centralized **group dispatcher** that assigns floor requests based on a cost-optimization strategy. Unlike conventional single-elevator controllers, this design introduces coordination between multiple elevators to improve efficiency, reduce waiting time, and balance load distribution.

Each elevator operates independently using a LOOK-based scheduling algorithm, allowing it to serve requests in an optimized directional sequence. The dispatcher evaluates multiple parameters such as distance, direction of movement, and current load to dynamically assign hall calls to the most suitable elevator. The entire system is implemented using a **single high-frequency clock**, with movement timing controlled via internal counters, ensuring efficiency and avoiding clock domain issues.

The design is verified through a comprehensive testbench simulating various real-world scenarios, including simultaneous requests, emergency interruptions, and full-load conditions. Additionally, a multiplexed 7-segment display interface is incorporated to visualize elevator positions in real time. This project demonstrates a scalable, modular, and hardware-efficient solution for elevator group control, showcasing practical applications of FSM design, digital system optimization, and Verilog-based hardware modeling.

2.Introduction

Elevator systems are an essential component of modern infrastructure, particularly in multi-story residential, commercial, and industrial buildings. As building heights increase and user demand becomes more dynamic, efficient elevator control systems become critical for minimizing waiting time, reducing energy consumption, and ensuring passenger safety. Traditional single-elevator control mechanisms are insufficient in handling complex traffic patterns, which has led to the development of **group control systems** involving multiple elevators working collaboratively.

This project focuses on the design of a **multi-elevator group control system** using Verilog HDL, with an emphasis on **finite state machine (FSM) modeling and efficient implementation**. FSMs are a fundamental concept in digital system design, widely used in control logic, communication protocols, and embedded systems. By implementing elevator behavior as an FSM, this project demonstrates a clear use of state transitions, output logic, and real-time decision-making in synchronous digital systems.

One of the key motivations behind this project is to bridge the gap between theoretical FSM concepts and real-world applications. Elevator control is a well-known problem in computer engineering and provides an excellent platform for demonstrating scheduling algorithms, concurrency, and hardware control. By designing separate FSMs for each elevator and integrating them with a centralized dispatcher, the project reflects a **modular and hierarchical design approach**, which is a core principle in digital system engineering.

A significant design decision in this project is the use of **Verilog HDL instead of microcontroller-based implementation**. While microcontrollers are commonly used for control systems due to their flexibility and ease of programming, they rely on sequential software execution and are limited by instruction cycle delays. In contrast, Verilog allows the design to be implemented directly (e.g., FPGA), enabling **true parallelism, deterministic timing, and higher performance**. This is particularly important for systems like elevator control, where multiple events must be handled simultaneously and reliably.

Others rely on simple scheduling techniques such as First-Come-First-Serve (FCFS), which can lead to inefficient movement and increased waiting times. Additionally, some implementations use multiple clocks or software-based delays, which can introduce synchronization issues and reduce hardware reliability.

This project addresses these limitations through several key improvements. First, it implements a **group dispatcher with a cost-based decision algorithm**, which considers distance, direction, and load to assign requests optimally. Second, it uses a **LOOK scheduling algorithm** within each elevator, ensuring efficient directional movement and minimizing unnecessary travel. Third, the design avoids multiple clock domains by using a **single clock with counter-based timing**,

improving robustness and synthesizability. Finally, the system is designed to be modular and scalable, allowing easy extension to more floors or additional elevators.

Another important feature of this project is the inclusion of **emergency handling**, where all elevator operations are safely halted upon receiving an emergency signal. This reflects real-world safety requirements and enhances the practical relevance of the design.

In summary, this project not only demonstrates the strong capability of FSM design and Verilog-based hardware modeling but also addresses real-world challenges in elevator group control. It combines theoretical knowledge with practical implementation, resulting in a system that is efficient, scalable, and aligned with modern digital design practices.

3 Design

3.1.1 Identification of scope

The scope of this project lies in the design and implementation of a **hardware-based multi-elevator control system** using Verilog HDL, with a focus on efficiency, modularity, and scalability. The system demonstrates how digital design principles can be applied to solve real-world control problems, particularly in the domain of building automation and smart infrastructure.

The importance of this project stems from its relevance to modern urban environments, where efficient vertical transportation systems are essential. Elevator group control systems are widely used in high-rise buildings to manage passenger flow and reduce waiting times. By implementing such a system at the hardware level, this project highlights the advantages of deterministic timing, parallel processing, and reliability, which are critical in safety-sensitive applications.

This design has several potential applications, including **residential and commercial building automation systems**, **industrial lift control**, and **smart city infrastructure**. It can also serve as a foundational model for more advanced systems incorporating features such as destination control, priority scheduling, and energy optimization. Furthermore, the project can be extended for use in **embedded FPGA-based controllers** in real-time environments.

In terms of implementation, the system is well-suited for deployment on FPGA platforms such as the Nexys A7 board. The modular structure allows easy expansion to support more floors or additional elevators by modifying parameters and extending the dispatcher logic. The use of a single clock domain ensures compatibility with standard digital design practices and simplifies hardware synthesis.

In conclusion, the scope of this project extends from academic demonstration to practical application in digital control systems. It provides a strong foundation for future work in hardware-based automation, intelligent control systems, and FPGA design, making it both educationally valuable and practically relevant.

3.1.2 Literature Review

Elevator group control systems have been an active area of research since the introduction of the first microprocessor-based elevator controller, the Elevonic 101, developed by the Otis Elevator Company in 1979 (Martin 2007; Fernandez and Cortes 2015). Since then, researchers have explored a wide range of approaches to improve the efficiency of vertical transportation, spanning from rule-based and fuzzy logic methods to evolutionary algorithms and, more recently, direct hardware implementations using programmable logic devices

Fernández et al. (2014) proposed the first entirely dynamic fuzzy logic elevator group control system designed to minimize passenger waiting time in high-rise buildings. The authors established that the elevator dispatching problem is NP-hard in nature, with the number of possible assignment solutions growing exponentially as the number of elevators and floors increases. Each possible elevator-floor assignment is evaluated through a weighted combination of two fuzzy inference criteria — an absolute time evaluation estimating actual passenger waiting time, and a relative time evaluation comparing one elevator's performance against all other available options for the same floor. However, the system requires continuous fuzzy inference computation and is implemented entirely in software, making real-time hardware deployment a significant challenge.

Kim et al. (1998) presented one of the earliest comprehensive fuzzy elevator group control systems, introducing a two-layer intelligent architecture that separates traffic pattern recognition from hall call assignment. The first layer classifies ongoing passenger traffic into one of eight predefined traffic modes using a fuzzy inference engine that evaluates five traffic features including the number of up-going and down-going passengers, traffic centralization ratios, and the current time of day. The classified traffic mode then determines the membership functions used in the second layer, which evaluates the suitability of each elevator for a given hall call across three criteria — average waiting time, long waiting percentage, and run count as a proxy for power consumption. These suitabilities are aggregated through an Ordered Weighted Average operator to select the most appropriate elevator.While effective, the system requires detailed traffic data collection, periodic learning, and complex membership function tuning, all of which impose significant computational and configuration overhead.

Bartz-Beielstein et al. introduced an algorithm-based validation methodology for a simplified elevator group controller model, addressing the fundamental challenge of how elevator control systems can be reliably tested and optimized without access to a real building (Bartz-Beielstein

et al. This work established an important methodological foundation for elevator control research, confirming that meaningful performance insights can be obtained through computationally tractable simplified models without the need for full-scale building simulations.

On the hardware implementation side, Varghese et al. (2015) presented an FPGA-based single elevator controller implemented in Verilog HDL and deployed on a Xilinx Artix-7 FPGA. The authors argued that traditional elevator controllers based on relay logic, PLCs, and microcontrollers suffer from a limited number of inputs and outputs and reduced flexibility, whereas FPGAs offer reconfigurability, low power consumption, fast response times, and scalable design — properties well-suited to the parallel and reactive nature of elevator control. The controller was designed as a Mealy finite state machine distinguishing between OPEN and MOVING states at each floor, with position sensors triggering state transitions upon physical arrival. However, the work addresses only a single elevator car and does not include any group dispatching logic for managing multiple elevators simultaneously.

Siddiqui and Hasan (2015) proposed a three-elevator parallel controller modeled in Verilog HDL and synthesized on a Spartan-3A FPGA using the Precision Synthesis Tool. The system controlled three elevators serving eight floors, with a common input console at every floor accepting hall call requests. The core dispatch logic was based on nearest elevator selection — upon receiving a hall call, the master controller evaluated the current positions of all three elevators and directed the closest one to respond, with the remaining two assigned lower priority. Each elevator followed a well-defined operational state flow from idle through travel, door opening, passenger boarding, destination travel, and return to idle, with dedicated clock circuits regulating door timing and an obstruction detection mechanism keeping doors open when a barrier is sensed. Despite demonstrating multi-elevator hardware control in Verilog, the nearest elevator selection strategy used does not account for the direction of travel, the number of floors already assigned to each elevator, or the penalty of forcing an elevator to reverse direction — all of which are important factors for efficient real-world group dispatching.

The reviewed works reveal a clear and consistent research gap. Software-based intelligent approaches such as fuzzy logic and metaheuristic algorithms achieve strong optimization performance but require substantial computational resources, training phases, and software infrastructure. Considering all these things makes direct hardware implementation difficult. On the other hand, existing Verilog and FPGA-based implementations demonstrate the viability of hardware elevator control but remain limited to simple nearest-car dispatch strategies without direction awareness, load balancing, or protection against hardware-specific issues such as elevator oscillation.

3.1.3 Formulation of problem

The central objective of this project is to design and implement a **multi-elevator group control system using Verilog HDL**, capable of efficiently handling passenger requests across multiple floors while ensuring optimal resource utilization, reliability, and scalability. The problem can be formulated as the development of a **hardware-based control mechanism** that coordinates multiple elevator cars in real time, minimizes passenger waiting time, and ensures safe and predictable operation under varying traffic conditions.

At its core, the challenge lies in **coordinating multiple independent agents (elevators)** to respond to a dynamic set of requests generated by users across different floors. Unlike a single-elevator system, where control logic is relatively straightforward, a group control system introduces complexity in terms of **decision-making, scheduling, and conflict resolution**. The system must determine not only how each elevator moves, but also which elevator should respond to a given request in order to achieve overall efficiency.

One of the primary challenges in this project was designing a system that can **balance responsiveness and efficiency**. A naive approach, such as assigning requests based solely on proximity or arrival order, can lead to inefficient movement patterns, unnecessary direction changes, and uneven load distribution among elevators. To address this, the project formulates the problem as an **optimization task**, where each request is evaluated based on multiple factors including distance, direction of movement, and current workload of each elevator. This led to the implementation of a **cost-based dispatching strategy**, allowing the system to make more informed and balanced decisions.

Another key challenge was ensuring **consistent and predictable behavior in practical scenarios**. Unlike many software-based systems, elevator group controller designs require strict adherence to timing constraints and synchronous operation. The project avoids the use of multiple clock domains, which are often a source of complexity and synchronization errors-often leading to high processing times due to multiple independent agents and clocks- and instead relies on a **single global clock with counter-based timing control**. This approach simplifies the design while maintaining precise control over elevator movement and state transitions.

The modeling of elevator behavior itself presented another layer of complexity. Each elevator must operate autonomously while still conforming to system-wide decisions. To achieve this, the behavior of each elevator is formulated as a **finite state machine (FSM)** with clearly defined states and transitions. This allows for structured and predictable control, ensuring that the elevator responds correctly to inputs such as new requests or emergency signals. The FSM-based design also reflects a fundamental advantage of digital system design principles, making it a suitable approach for implementation in Verilog.

A significant challenge encountered in the project was the **management of dynamic and asynchronous requests**. Passenger inputs can occur at any time and may overlap or conflict with

existing assignments. To handle this, the system incorporates a mechanism for **tracking and maintaining pending requests**, ensuring that no request is lost or prematurely cleared. Additionally, the concept of **"sticky assignment"** is introduced, where once a request is assigned to an elevator, it remains associated with that elevator until it is served or withdrawn. This prevents instability and repeated reassignment, which could otherwise degrade system performance.

Another important aspect of the problem formulation is the handling of **directional movement and scheduling within each elevator**. Simply moving to the nearest request is not always optimal, especially when multiple requests are distributed across different floors. To address this, the project adopts a **direction-based scheduling strategy inspired by the LOOK algorithm**, where each elevator continues servicing requests in its current direction before reversing. This reduces unnecessary travel and improves overall efficiency, particularly under moderate to high traffic conditions.

Safety considerations also play a crucial role in the system design. The inclusion of an **emergency stop mechanism** introduces additional constraints, as the system must immediately halt all movement and ignore ongoing operations when an emergency signal is triggered. This requires careful handling within both the FSM and the dispatcher to ensure that the system can safely transition to a halted state and later resume normal operation without inconsistencies.

From an implementation perspective, another challenge was ensuring that the design remains **modular and scalable**. The project is structured in such a way that individual components, such as the elevator controllers and dispatcher, can be modified or extended independently. This modularity not only simplifies debugging and testing but also allows the system to be adapted for larger configurations, such as increasing the number of floors or elevators.

In terms of novelty, this project introduces several features that distinguish it from simpler or conventional implementations. These include the integration of a **multi-parameter cost function for dispatching**, the use of a **LOOK-based scheduling algorithm within each elevator**, and the adoption of a **single-clock, counter-driven timing mechanism** for hardware efficiency. Additionally, the design emphasizes **balanced load distribution and stable request handling**, which are often overlooked in basic elevator control models.

The methods used in this project combine principles from digital logic design, control systems, and algorithmic optimization. FSM modeling provides a structured framework for control, while the cost-based approach introduces an element of decision-making that enhances system performance. The use of parameterization further improves flexibility, allowing key aspects of the design to be adjusted without significant restructuring.

In summary, the formulation of this problem revolves around designing a **quick and efficient, scalable, and intelligent elevator group control system** that effectively manages multiple inputs, optimizes movement, and ensures safe operation. The challenges encountered during the

project have been addressed through careful design choices, resulting in a system that demonstrates both astute decision handling capability and practical applicability in digital control systems.

3.1.4 Analysis

The proposed elevator group controller is grounded in the classical theory of digital sequential circuits, specifically the concept of a **Finite State Machine (FSM)**. A finite state machine is a mathematical model of computation that describes a system which can exist in exactly one of a finite number of defined states at any given moment. The system transitions from one state to another in response to inputs, and produces outputs based on its current state or a combination of its state and inputs. FSMs are one of the most fundamental building blocks of digital hardware design, and they are particularly well-suited to control systems where behavior must be predictable.

There are two classical types of finite state machines. In a **Mealy machine**, the outputs depend on both the current state and the current inputs, meaning outputs can change immediately when an input changes even without a clock edge. In a **Moore machine**, the outputs depend only on the current state. Moore machines are generally preferred in synchronous digital hardware design because their outputs are registered and stable, they are easier to verify through simulation, they have no combinational glitch paths from input to output. **Moore machines** are simple to integrate cleanly with the synchronous clocked design methodology used in FPGAs.

In this project, the lift controller module is implemented as a **synchronous Moore FSM**. The four states — IDLE, MOVE_UP, MOVE_DOWN, and EMERGENCY. These four states fully determine the three output signals move_up, move_down and motor_stop without any dependence on current inputs. The next-state logic is computed combinationally from the current state and inputs. However, the state register itself is updated only on the rising edge of the clock, ensuring that all state transitions are synchronous and all outputs are glitch-free. This design choice directly improves the reliability and predictability of the motor control signals. This design can play critical role in a safety-relevant application such as an elevator.

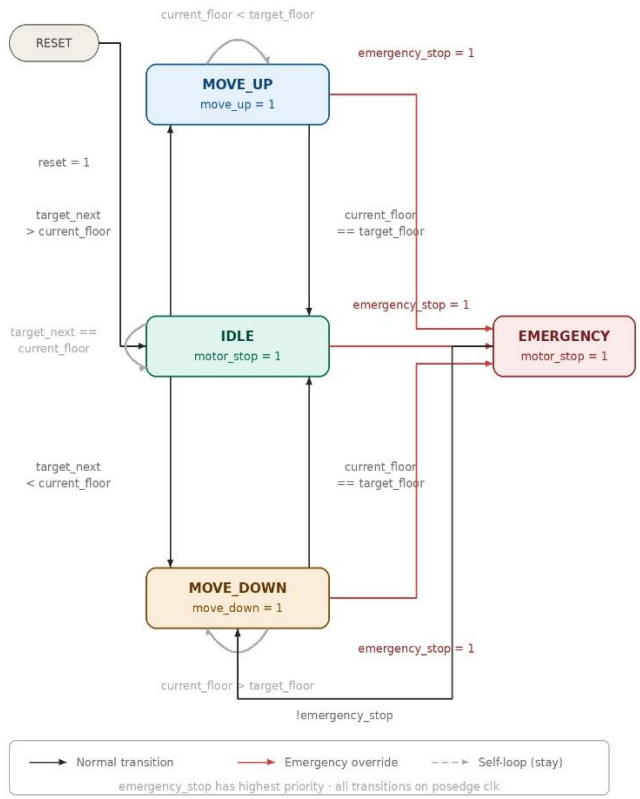


Figure: The state diagram of a single lift controller

The dispatcher module uses a robust logic for calculating costs for both the lift cars to reach a number of floors when a hall call is made. The cost calculation algorithm contains distance, direction and load balancing features. The dispatcher works as the main selector of the system which chooses the most efficient lift to serve a floor.

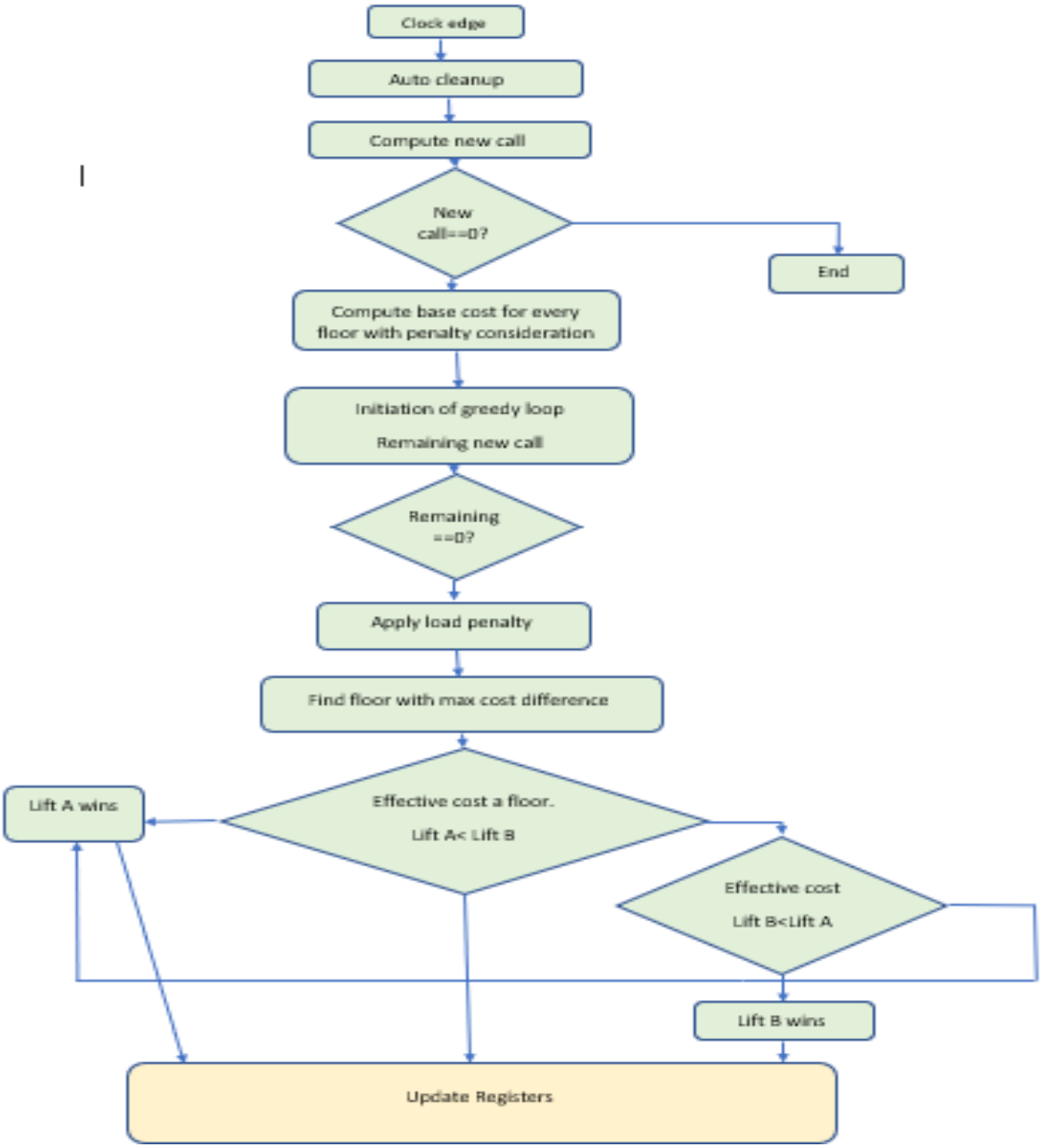


Figure: Dispatcher module flow chart

The overall system architecture follows a **hierarchical modular design** philosophy. The top-level module instantiates two independent lift controller modules and one group dispatcher module, connecting them through a well-defined set of signals. This separation of concerns means that the movement logic of each car is completely encapsulated within its own controller, while the dispatch intelligence resides entirely in the dispatcher. The two levels communicate through the assigned request and request for individual lift car signals respectively, creating a clean producer-consumer interface between the dispatcher and the controllers.

3.2 Design Method

The design implements a **two-car elevator group controller** for a four-floor building using Verilog HDL. It is organized into three distinct modules. The lift controller module governs the physical movement logic of a single lift car, the group dispatcher module handles intelligent call assignment between the two cars, and the top group controller module integrates everything into a unified, synthesizable top-level design with seven-segment display output.

The **lift _controller module** is parameterized with MOVE_DELAY , which determines how many clock cycles the car spends traveling between consecutive floors — defaulting to 100,000,000 cycles, corresponding to one second on a 100 MHz clock. The lift car operates as a **four-state Moore finite state machine** with the states given below:

- **IDLE** :The car is stationary, awaiting a new target.
- **MOVE_UP** : The car is traveling upward toward its target floor.
- **MOVE_DOWN** : The car is descending toward its target floor.
- **EMERGENCY** : All motion is immediately halted upon assertion of this state. The car returns to IDLE only when the signal is de-asserted.

State transitions are registered on the positive clock edge, while the combinational next_state logic evaluates conditions such as arrival at the target floor and the emergency signal.

A 25-bit countdown register **move_cnt** enforces the inter-floor travel time. It reloads to **MOVE_DELAY - 1** on reset and upon expiry, producing a single-cycle **move_enable** pulse that authorizes the **current_floor** register to increment or decrement by one. This ensures the car advances exactly one floor per **MOVE_DELAY** clock cycles.

The car maintains a 4-bit **pending_req** register, where each bit represents a floor that has been assigned to this car and remains unserved. New floors are latched from **assigned_req** — which comes from the dispatcher's output into **pending_req** each clock cycle.

Target selection follows the **LOOK scheduling algorithm**. Rather than scanning to a physical end-stop, the car reverses direction when no further requests exist in the current direction of travel. The combinational block evaluates **pending_req** and **last_dir** to produce **target_next** and **dir_next**, which are registered into **target_floor** and **last_dir** respectively whenever the car is in the IDLE state.

The most critical design decision in this module addresses a subtle but severe **oscillation bug**. The module introduces a 4-bit **already_served** register. When the car arrives at a floor, **already_served[current_floor]** is set to 1. The latching condition for **pending_req** then becomes a three-way conjunction:

This means a floor is latched only if it is assigned, **not already served this cycle**, and is not the lift car's current position. The **already_served** bit is cleared only when **assigned_req[f]** drops when the hall call for that floor is officially released and resetting the slot for legitimate future requests.

This **group_dispatcher** module acts as the brain of the group controller. It monitors all incoming hall calls and decides, in real time, which car should serve each call. It ensures the parallel operation of both the lift cars.

A **served_call** register tracks which hall calls have already been dispatched in the current active call cycle. This prevents the dispatcher from re-dispatching a floor that is already being handled by one of the cars while its hall call button remains pressed. Both **served_call** and the **req_A/req_B** outputs are masked with **hall_call** each cycle, so that when a call is cleared the assignment is cleanly withdrawn.

For each floor and each car, the dispatcher computes a **cost metric** composed of two components:

- **Distance cost** — the absolute difference between the car's current floor and the target floor, computed via the **absdiff** function.
- **Directional penalty** — assessed via the **dir_penalty** function:
    1. **0** if the car is already moving toward the target (natural direction of travel).
    2. **1** if the car is idle, but the target floor is in the same direction of the last direction stored in the lift car memory.
    3. **3** if the car is idle, but the target floor is in the opposite direction of the last direction stored in the lift car memory.
    4. **4** if the car would have to reverse direction to serve the call (strongly discouraged).

This penalty parameters has been selected after trail and error all the possible cases to ensure to make the system related to real life elevator activities.

Rather than assigning all calls simultaneously, the dispatcher uses a **greedy iterative strategy over up to four passes**:

- In each pass, it identifies the floor among the remaining unassigned calls where the **cost differential** between the two cars is greatest — i.e., where one car has the clearest advantage.
- That floor is assigned first, to the car with the lower effective cost.

- Once a car receives an assignment, a load penalty of **+10** is added to its effective cost for all subsequent floors in the same dispatch cycle. This strongly discourages assigning a second floor to the same car when the other car is available, promoting balanced, parallel utilization.
- In case of a tie, the car with the lower current load is preferred.

This approach mirrors the behavior of practical group controller algorithms used in commercial elevator systems, achieving near-optimal dispatch without exhaustive search.

The group_controller_top module is the synthesizable top-level integration module. It instantiates two lift_controller units (Lift Car A and Lift Car B) and one group_dispatcher, wiring them together with appropriate signal routing. The floor_A and floor_B outputs from the controllers feed back into the dispatcher, enabling real-time position-aware dispatching.

The module finally drives a multiplexed **two-digit seven-segment display** to show the current floors of both cars simultaneously. A 15-bit refresh counter cycles continuously, and its most significant bit alternates the active digit at a rate well within the persistence of vision threshold (~1–2 kHz), making both digits appear simultaneously lit to the human eye. The an register selects the active digit, while seg is driven with the appropriate floor encoding (0–3 in standard seven-segment patterns). The decimal point is permanently disabled.

3.1 Full Source Code of Firmware

Verilog Code:

```
`timescale 1ns/1ps

module lift_controller #(

    parameter MOVE_DELAY = 100_000_000

)(

    input        clk,

    input        reset,

    input  [4:0] assigned_req,

    input        emergency_stop,

    output reg   move_up,

    output reg   move_down,

    output reg   motor_stop,

    output reg   [2:0] current_floor,

    output wire        last_dir_out

);

    localparam IDLE      = 2'b00;

    localparam MOVE_UP   = 2'b01;

    localparam MOVE_DOWN = 2'b10;

    localparam EMERGENCY = 2'b11;



    reg [1:0]  current_state, next_state;

    reg [24:0] move_cnt;

    wire       move_enable = (move_cnt == 25'd0);



    always @(posedge clk or posedge reset) begin

        if (reset)            move_cnt <= MOVE_DELAY -
1;

        else if (move_enable) move_cnt <= MOVE_DELAY -
1;

        else                  move_cnt <= move_cnt -
1'b1;

    end



    reg       last_dir;

    reg [4:0] pending_req;

    reg [2:0] target_floor;

    reg [4:0] already_served;
```

```
`timescale 1ns / 1ps

module group_dispatcher (

    input        clk,

    input        reset,

    input  [4:0] hall_call,

    input  [2:0] floor_A,

    input  [2:0] floor_B,

    input        move_up_A,

    input        move_up_B,

    input        move_down_A,

    input        move_down_B,

    input        stop_A,

    input        stop_B,

    input        last_dir_A,

    input        last_dir_B,

    output reg [4:0] req_A,

    output reg [4:0] req_B

);

    reg [4:0] served_call;

    wire [4:0] new_call = hall_call & ~served_call;



    function [2:0] absdiff;

        input [2:0] a, b;

        begin absdiff=(a>=b)?(a-b):(b-a); end

    endfunction



    function [2:0] dir_penalty;

        input [2:0] car_floor;

        input [2:0] target;

        input       going_up;

        input       going_down;

        input       idle;

        input       last_dir;

        begin

            if (!idle) begin
```

```
    assign last_dir_out = last_dir;



    always @(posedge clk or posedge reset) begin

        if (reset) begin

            pending_req    <= 5'b00000;

            already_served <= 5'b00000;

        end else begin

            already_served <= already_served &
assigned_req;

            pending_req <= pending_req

                        | (assigned_req

                           & ~already_served

                           & ~(5'b00001 <<
current_floor));

            if (next_state == IDLE) begin

                pending_req[current_floor]    <= 1'b0;

                already_served[current_floor] <= 1'b1;

            end

        end

    end



    reg [2:0] target_next;

    reg       dir_next;



    always @(*) begin

        target_next = current_floor;

        dir_next    = last_dir;

        if (!emergency_stop) begin

            if (last_dir == 1'b1) begin

                if (pending_req[1] && current_floor <
3'd1)

                    begin target_next=3'd1;
dir_next=1'b1; end

                else if (pending_req[2] &&
current_floor < 3'd2)

                    begin target_next=3'd2;
dir_next=1'b1; end

                if (going_up && target>car_floor)

                    dir_penalty=3'd0;

                else if (going_down && target<car_floor)

                    dir_penalty=3'd0;

                else dir_penalty=3'd4;

            end else begin

                if (last_dir==1'b1) begin

                    if (target>car_floor)
dir_penalty=3'd1;

                    else
dir_penalty=3'd3;

                end else begin

                    if (target<car_floor)
dir_penalty=3'd1;

                    else
dir_penalty=3'd3;

                end

            end

        end

    endfunction



    reg [2:0] base_cost_A [0:4];

    reg [2:0] base_cost_B [0:4];

    integer   i;



    always @(*) begin

        for (i=0; i<5; i=i+1) begin

            base_cost_A[i]=absdiff(floor_A,i[2:0])

                +dir_penalty(floor_A,i[2:0],


move_up_A,move_down_A,stop_A,last_dir_A);

            base_cost_B[i]=absdiff(floor_B,i[2:0])

                +dir_penalty(floor_B,i[2:0],


move_up_B,move_down_B,stop_B,last_dir_B);

        end

    end
```

```
                else if (pending_req[3] &&
current_floor < 3'd3)

                    begin target_next=3'd3;
dir_next=1'b1; end

                else if (pending_req[4] &&
current_floor < 3'd4)

                    begin target_next=3'd4;
dir_next=1'b1; end

                else if (pending_req[3] &&
current_floor > 3'd3)

                    begin target_next=3'd3;
dir_next=1'b0; end

                else if (pending_req[2] &&
current_floor > 3'd2)

                    begin target_next=3'd2;
dir_next=1'b0; end

                else if (pending_req[1] &&
current_floor > 3'd1)

                    begin target_next=3'd1;
dir_next=1'b0; end

                else if (pending_req[0] &&
current_floor > 3'd0)

                    begin target_next=3'd0;
dir_next=1'b0; end

            end else begin

                if (pending_req[3] && current_floor >
3'd3)

                    begin target_next=3'd3;
dir_next=1'b0; end

                else if (pending_req[2] &&
current_floor > 3'd2)

                    begin target_next=3'd2;
dir_next=1'b0; end

                else if (pending_req[1] &&
current_floor > 3'd1)

                    begin target_next=3'd1;
dir_next=1'b0; end

                else if (pending_req[0] &&
current_floor > 3'd0)

                    begin target_next=3'd0;
dir_next=1'b0; end

                else if (pending_req[1] &&
current_floor < 3'd1)

                    begin target_next=3'd1;
dir_next=1'b1; end

                else if (pending_req[2] &&
current_floor < 3'd2)
```

```
    always @(posedge clk or posedge reset) begin

        if (reset) begin

            req_A<=5'b00000; req_B<=5'b00000;

            served_call<=5'b00000;

        end else begin

            served_call<=served_call & hall_call;

            req_A<=req_A & hall_call;

            req_B<=req_B & hall_call;

            if (new_call!=5'b00000) begin :
dispatch_block

                integer load_A,load_B,best_i,best_diff;

                integer curr_diff,eff_A,eff_B,pass,j;

                reg [4:0] remaining;

                load_A=0; load_B=0; remaining=new_call;

                for (pass=0; pass<5; pass=pass+1) begin

                    if (remaining!=5'b00000) begin

                        best_i=-1; best_diff=-1;

                        for (j=0; j<5; j=j+1) begin

                            if (remaining[j]) begin


eff_A=base_cost_A[j]+(load_A>0?10:0);


eff_B=base_cost_B[j]+(load_B>0?10:0);

                                curr_diff=(eff_A>eff_B)

                                    ?(eff_A-
eff_B):(eff_B-eff_A);

                                if (curr_diff>best_diff)
begin

                                    best_diff=curr_diff;
best_i=j;

                                end

                            end

                        end

                        if (best_i>=0) begin


eff_A=base_cost_A[best_i]+(load_A>0?10:0);


eff_B=base_cost_B[best_i]+(load_B>0?10:0);

                            if (eff_A<eff_B) begin

                                req_A[best_i]<=1'b1;
```

```
                    begin target_next=3'd2;
dir_next=1'b1; end

                else if (pending_req[3] &&
current_floor < 3'd3)

                    begin target_next=3'd3;
dir_next=1'b1; end

                else if (pending_req[4] &&
current_floor < 3'd4)

                    begin target_next=3'd4;
dir_next=1'b1; end

            end

        end

    end



    always @(posedge clk or posedge reset) begin

        if (reset) begin

            target_floor <= 3'd0;

            last_dir     <= 1'b1;

        end else if (current_state == IDLE) begin

            target_floor <= target_next;

            if (current_floor == 3'd4)      last_dir <=
1'b0;

            else if (current_floor == 3'd0) last_dir <=
1'b1;

            else                            last_dir <=
dir_next;

        end

    end



    always @(posedge clk or posedge reset) begin

        if (reset) current_state <= IDLE;

        else       current_state <= next_state;

    end



    always @(posedge clk or posedge reset) begin

        if (reset) current_floor <= 3'd0;

        else if (move_enable) begin

            if (current_state==MOVE_UP &&
current_floor<target_floor)
```

```
served_call[best_i]<=1'b1;

                                load_A=load_A+1;

                            end else if (eff_B<eff_A)
begin

                                req_B[best_i]<=1'b1;


served_call[best_i]<=1'b1;

                                load_B=load_B+1;

                            end else begin

                                if (load_A<=load_B)
begin

                                    req_A[best_i]<=1'b1;


served_call[best_i]<=1'b1;

                                    load_A=load_A+1;

                                end else begin

                                    req_B[best_i]<=1'b1;


served_call[best_i]<=1'b1;

                                    load_B=load_B+1;

                                end

                            end

                            remaining[best_i]=1'b0;

                        end

                    end

                end

            end

        end

    end

endmodule



`timescale 1ns / 1ps

module group_controller_top #(

    parameter MOVE_DELAY = 100_000_000

)(

    input        clk, reset,

    input  [4:0] sw_hall,

    input        btn_commit,
```

```
                current_floor <= current_floor + 1'b1;

            else if (current_state==MOVE_DOWN &&
current_floor>target_floor)

                current_floor <= current_floor - 1'b1;

        end

    end



    always @(*) begin

        next_state = current_state;

        if (emergency_stop) next_state = EMERGENCY;

        else case (current_state)

            IDLE:      if (target_next>current_floor)
next_state=MOVE_UP;

                       else if
(target_next<current_floor) next_state=MOVE_DOWN;

            MOVE_UP:   if (current_floor==target_floor)
next_state=IDLE;

            MOVE_DOWN: if (current_floor==target_floor)
next_state=IDLE;

            EMERGENCY: if (!emergency_stop)
next_state=IDLE;

            default:   next_state = IDLE;

        endcase

    end



    always @(*) begin

        move_up=1'b0; move_down=1'b0; motor_stop=1'b0;

        case (current_state)

            MOVE_UP:   move_up    = 1'b1;

            MOVE_DOWN: move_down  = 1'b1;

            default:   motor_stop = 1'b1;

        endcase

    end

endmodule
```

```
    input        emergency_stop,

    output wire  move_up_A, move_down_A, motor_stop_A,

    output wire  move_up_B, move_down_B, motor_stop_B,

    output reg  [6:0] seg,

    output wire        dp,

    output reg  [7:0] an

);

    wire [2:0] floor_A, floor_B;

    wire [4:0] req_A,   req_B;

    wire move_up_A_w,move_down_A_w,stop_A_w;

    wire move_up_B_w,move_down_B_w,stop_B_w;

    wire last_dir_A, last_dir_B;



    reg [4:0] hall_call;

    reg       btn_prev;

    wire btn_rising = btn_commit & ~btn_prev;



    always @(posedge clk or posedge reset) begin

        if (reset) begin

            hall_call<=5'b00000; btn_prev<=1'b0;

        end else begin

            btn_prev<=btn_commit;

            if (btn_rising) hall_call<=sw_hall;

        end

    end



    lift_controller #(.MOVE_DELAY(MOVE_DELAY)) car_A (

        .clk(clk),.reset(reset),

        .assigned_req(req_A),

        .emergency_stop(emergency_stop),

        .move_up(move_up_A),.move_down(move_down_A),

        .motor_stop(motor_stop_A),

        .current_floor(floor_A),

        .last_dir_out(last_dir_A));

    assign move_up_A_w=move_up_A;
```

```
    assign move_down_A_w=move_down_A;
    assign stop_A_w=motor_stop_A;


    lift_controller #(.MOVE_DELAY(MOVE_DELAY)) car_B (
        .clk(clk),.reset(reset),
        .assigned_req(req_B),
        .emergency_stop(emergency_stop),
        .move_up(move_up_B),.move_down(move_down_B),
        .motor_stop(motor_stop_B),
        .current_floor(floor_B),
        .last_dir_out(last_dir_B));
    assign move_up_B_w=move_up_B;
    assign move_down_B_w=move_down_B;
    assign stop_B_w=motor_stop_B;


    group_dispatcher dispatcher (
        .clk(clk),.reset(reset),
        .hall_call(hall_call),
        .floor_A(floor_A),.floor_B(floor_B),
        .move_up_A(move_up_A_w),.move_up_B(move_up_B_w),
.move_down_A(move_down_A_w),.move_down_B(move_down_B_w),
        .stop_A(stop_A_w),.stop_B(stop_B_w),
        .last_dir_A(last_dir_A),.last_dir_B(last_dir_B),
        .req_A(req_A),.req_B(req_B));


    reg [15:0] refresh_cnt;
    always @(posedge clk or posedge reset)
        if (reset) refresh_cnt<=16'd0;
        else       refresh_cnt<=refresh_cnt+1'b1;


    wire [1:0] slot=refresh_cnt[15:14];


    function [6:0] floor_seg;
        input [2:0] floor;
```

```
        begin
            case(floor)
                3'd0: floor_seg=7'b000_0001;
                3'd1: floor_seg=7'b100_1111;
                3'd2: floor_seg=7'b001_0010;
                3'd3: floor_seg=7'b000_0110;
                3'd4: floor_seg=7'b100_1100;
                default: floor_seg=7'b111_1111;
            endcase
        end
    endfunction



    always @(*) begin
        case(slot)
            2'b00: begin seg=7'b000_1000;
an=8'b0111_1111; end
            2'b01: begin seg=floor_seg(floor_A);
an=8'b1110_1111; end
            2'b10: begin seg=7'b1100000;
an=8'b1111_0111; end
            2'b11: begin seg=floor_seg(floor_B);
an=8'b1111_1110; end
            default: begin seg=7'b111_1111;
an=8'b1111_1111; end
        endcase
    end
    assign dp=1'b1;
endmodule
```

`

Table: Full Verilog Source Code

4. Implementation

The Elevator Group Controller system was implemented in a FPGA board NEXYS A7 100T using Verilog HDL to manage the operation of two elevators across five floors. The system accepts input requests from each floor and processes them using a cost-based decision algorithm. The controller calculates the “cost” (based on distance, direction, and current state of each elevator) and assigns the request to the elevator with the lowest cost.

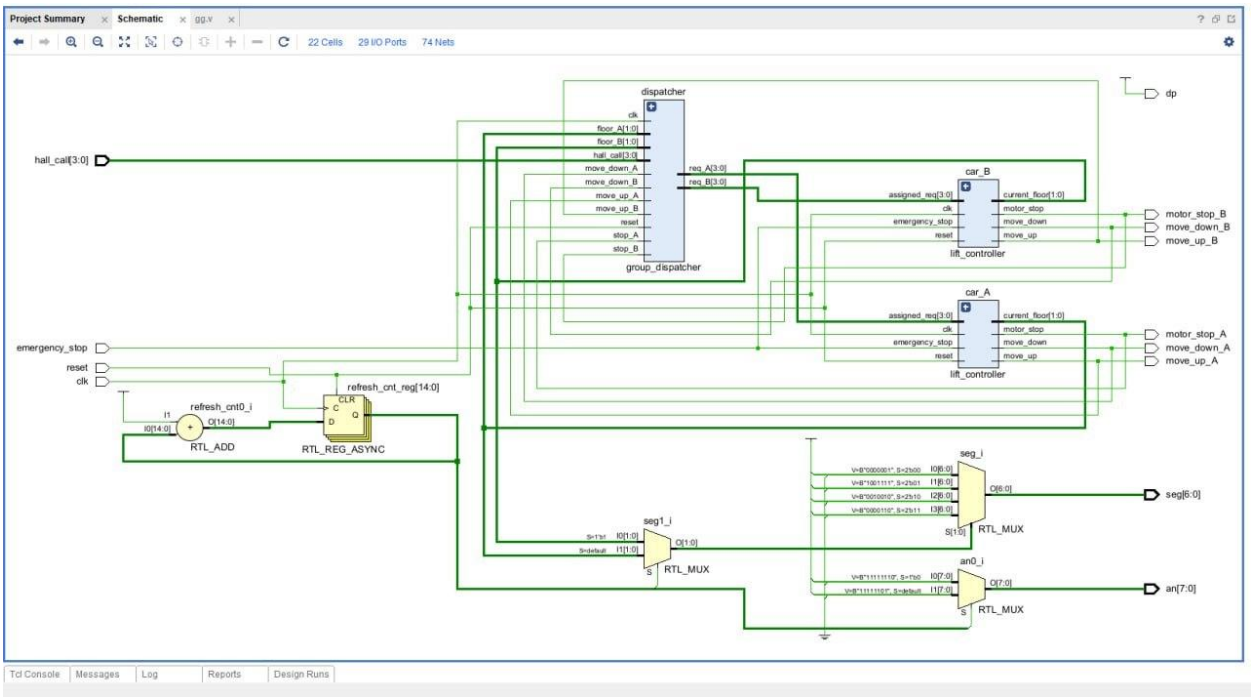


Fig : Schematic Diagram

Each elevator module maintains its own state, including current floor, movement direction, and request queue. A centralized control logic evaluates incoming requests and dynamically updates elevator assignments. The design was simulated and verified using a hardware description simulation tool to ensure correct logical behavior under multiple request scenarios.

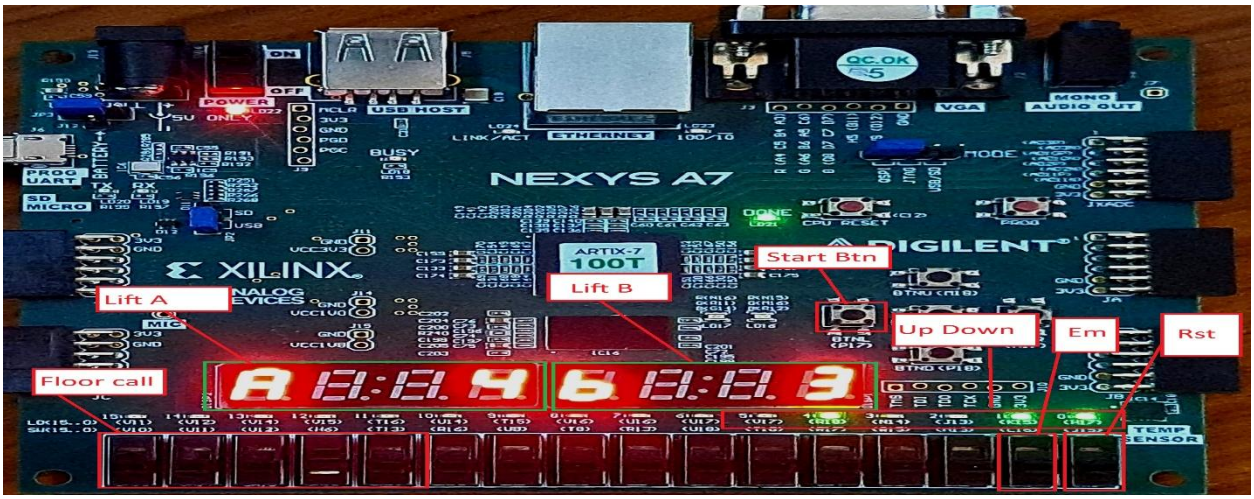


Fig : Used switches, buttons, leds & 7 segment of FPGA board

5.Design Analysis and Evaluation

5.1 Novelty

- The system uses a **cost-based decision algorithm** instead of simple first-come-first-serve scheduling.
- It dynamically assigns elevators based on **real-time conditions**, improving efficiency.
- The design supports **multi-elevator coordination**, which is more advanced than single-elevator controllers.

5.2 Design Considerations

5.2.1 Considerations to Public Health and Safety

The system is designed with a strong focus on safety, ensuring that elevator operations are reliable and efficient. One of the primary safety features is the prevention of conflicting commands, such as multiple elevators simultaneously responding to the same request, which can lead to delays, confusion, and potentially hazardous situations. By implementing advanced sequencing logic, the system ensures that only the most appropriate elevator responds to a given request, thereby minimizing the risk of unnecessary movements and reducing the chance of operational errors. Furthermore, the system's efficient coordination between elevators helps avoid abrupt stops, sudden direction changes, or unsafe acceleration, which are critical factors in preventing accidents or discomfort for passengers. In addition to this, the system is designed to handle emergency situations swiftly, ensuring that elevator operations can be safely halted when necessary and resumed in a controlled manner. This comprehensive approach enhances overall

passenger safety, ensuring that the elevators operate within the required safety standards at all times.

5.2.2 Considerations to Environment

The system's efficient scheduling and resource allocation are key factors in reducing the environmental impact of elevator systems. By minimizing unnecessary elevator movements, the system not only cuts down on energy consumption but also helps lower the wear and tear on mechanical components, leading to a longer lifespan of the equipment. Unnecessary trips and idling are minimized through the intelligent allocation of requests to the most appropriate elevator, ensuring that elevators only move when required, thereby conserving energy. This efficiency is particularly significant in large buildings, where elevator systems can be major consumers of power. Reducing energy usage directly contributes to a decrease in the building's overall carbon footprint, supporting environmentally sustainable practices. By optimizing the elevator operation, the system helps align with broader environmental goals, including energy conservation, lower emissions, and the promotion of green building initiatives, making it an important contribution to eco-friendly building operations.

5.2.3 Considerations to Cultural and Societal Needs

The design of the elevator control system prioritizes user convenience, making it an essential component in modern buildings where time is a valuable resource. By reducing waiting times and optimizing the scheduling of elevator responses, the system greatly enhances the overall user experience, especially in high-traffic buildings such as commercial centers, residential complexes, and office buildings. The ability to serve requests quickly and efficiently is a critical societal need, ensuring that people can access their desired floors with minimal delay. Additionally, the system takes into account the need for equitable service, ensuring that all users have access to the elevator system in a fair and non-discriminatory manner. Instead of relying on random or ad-hoc assignment of requests, the system uses a logical approach to prioritize elevator assignments based on various factors such as floor distance, current load, and direction, ensuring that all users are treated fairly and efficiently. This consideration of societal needs makes the system not only practical but also aligned with modern values of inclusivity, accessibility, and enhanced public services.

5.3 Investigations

5.3.1 Design of Experiment

Multiple simulation scenarios were designed, including:

- Single request cases
- Multiple simultaneous requests
- Opposite direction requests
- Idle and busy elevator conditions

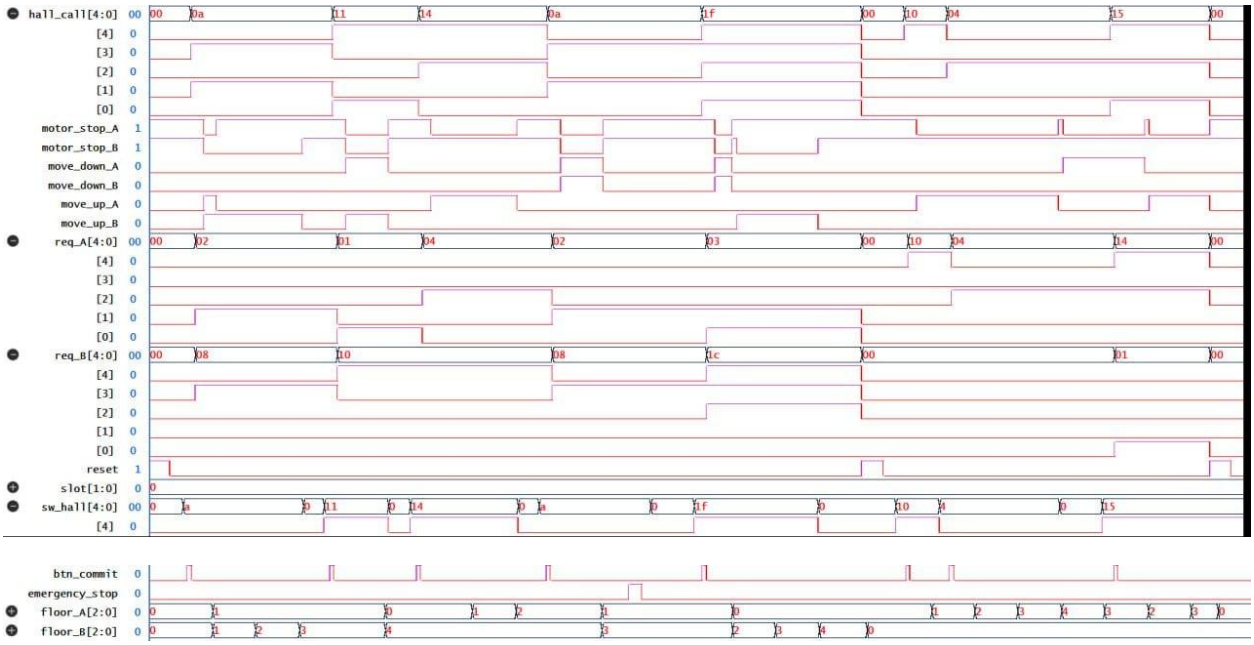
5.3.2 Data Collection

Simulation outputs were recorded, including:

- Elevator response time
- Floor assignment decisions
- Movement patterns

5.3.3 Results and Analysis

The timing diagram of total 8 cases obtained from the EDA PLAYGROUND are given below:



The result of the timing diagram is summerised in the table below.

PHASE	TEST SCENARIO	HALL CALL (FLOORS)	CAR A RESULT	CAR B RESULT	STATUS
Phase 0 — Reset behaviour					
PH0	System reset — both cars initialised	`00000`	Floor 0, motor stopped	Floor 0, motor stopped	PASS
Phase 1 — Basic two-floor dispatch					
PH1	Dispatch to floors 1 and 3 — cars split correctly	`01010`	Reached floor 1	Reached floor 3	PASS
Phase 2 — Extremes dispatch (floors 0 and 4)					
PH2	Cars at floors 1 & 3 sent to opposite ends	`10001`	Reached floor 0	Reached floor 4	PASS
Phase 3 — Partial call (floors 2 and 4)					
PH3	Car B stays at floor 4, Car A moves to floor 2	`10100`	Reached floor 2	Stayed at floor 4	PASS
Phase 4 — Emergency stop during travel					
PH4-a	Emergency asserted — both cars halt immediately	`01010`	Stopped at floor 1, move_up = 0	Stopped at floor 3, move_up = 0	PASS
PH4-b	Emergency released — cars return to idle	`01010`	Idle at floor 1	Idle at floor 3	PASS
Phase 5 — All-floor dispatch (load balancing)					
PH5	All 5 floors called — no floor assigned to both cars	`11111`	Floors 0, 1 assigned (reqA = 00011)	Floors 2, 3, 4 assigned (reqB = 11100)	PASS
Phase 6 — Dynamic mid-journey dispatch					
PH6	Floor 2 call added while Car A travelling to floor 4	`00100`	Accepted floor 2 en-route, reached floor 4	Remained at floor 0	PASS
Phase 7 — Reset during active call					
PH7	Reset asserted while cars in motion with pending requests	`10101`	Floor 0, stopped, req_A cleared	Floor 0, stopped, req_B cleared	PASS

The results are consistent with our algorithm. Each time the the hall calls were dispatched to the nearest lift car. In case of emergency stop, the lift car immediately stop wherever they are. When the whole system was set to RESET, all the lift cars descended to floor 0 according to the timing diagram result. Besides, parallel operation of both the lift cars is seen in the timing diagram. So,

it can be said that our design has met the objective of the project. We also tested this cases on the FPGA board NEXYS 7-100T, the results were the same as the timing diagram.

5.3.4 Interpretation and Conclusions on Data

The controller performs efficiently under varying conditions. It demonstrates improved performance in handling multiple requests, confirming the effectiveness of the design.

5.4 Limitations of Tools

- Simulation tools cannot fully replicate real-world hardware constraints.
- Timing delays and physical system dynamics are not perfectly modeled.
- Our designed model is solely focused on the lift movement of a certain hall call by the users.
- It internally considers the direction movement as a factor, but our design does not incorporate it explicitly like real life “UP” and “DOWN” button on each floor like a practical elevator.
- Besides, the internal floor switches in the elevator has not been included in the design.

5.5 Impact Assessment

5.5.1 Assessment of Societal and Cultural Issues

The system significantly enhances the user experience in both public and residential buildings by effectively addressing common pain points such as long waiting times and unreliable service. In high-traffic areas like shopping malls, office buildings, and residential complexes, users often face frustration due to delayed elevator service, especially during peak hours. By optimizing elevator operations, this system reduces waiting time, ensuring that users experience faster and more efficient service. Additionally, it increases the reliability of the service, as the system minimizes the occurrence of malfunctions and operational delays. In residential buildings, particularly in high-rise complexes, where elevator congestion is a regular concern, this system fosters a more convenient and stress-free living environment. Beyond mere functionality, the system's ability to enhance convenience plays a significant role in improving overall satisfaction, contributing positively to the social dynamics of shared spaces and promoting a greater sense of comfort and well-being for all residents and visitors.

5.5.2 Assessment of Health and Safety Issues

The system plays a vital role in ensuring the health and safety of elevator users by maintaining proper scheduling and avoiding conflicts between car assignments. One of the most significant risks in elevator operation arises from overcrowding or erratic scheduling, which can lead to

dangerous situations such as sudden stops, uneven load distribution, or even system malfunctions. By incorporating sophisticated algorithms for scheduling, the system ensures that elevators are used optimally without overwhelming the capacity, which helps to prevent potential safety hazards. For example, the system can manage peak traffic periods efficiently, ensuring smooth transitions between floors without abrupt starts or stops that could cause discomfort or accidents. Furthermore, by prioritizing certain floors or restricting access during critical times, it helps avoid bottlenecks, reducing the likelihood of accidental injuries caused by overcrowding or user impatience. By proactively addressing these concerns, the system significantly contributes to safer elevator operation, enhancing the overall health and safety standards within buildings.

5.5.3 Assessment of Legal Issues

The design and implementation of the elevator system must adhere to a variety of legal and regulatory requirements to ensure the safety and well-being of its users. In real-world applications, particularly in commercial or residential buildings, the system must comply with local building safety codes, national elevator standards, and international safety regulations. These rules govern everything from mechanical design and load capacities to emergency protocols and accessibility features. The current prototype, while demonstrating promising functionality, does not yet include the necessary regulatory certifications required for full implementation in operational systems. Before the system can be deployed in actual buildings, it will need to undergo rigorous testing and certification processes to verify its compliance with safety standards. This involves detailed inspections by regulatory bodies to ensure that the system does not pose any risks to users and aligns with the legal expectations surrounding elevator technology. As such, while the prototype shows great potential, it remains subject to further development and approval to meet all necessary legal and safety requirements before its commercial rollout.

5.6 Sustainability Evaluation

The system plays a pivotal role in fostering sustainability by optimizing the usage of elevator systems, ensuring that each trip is purposeful and necessary. By minimizing unnecessary movements, the system significantly reduces energy consumption, which not only lowers operational costs but also contributes to the overall reduction in the building's carbon footprint. This efficiency extends to reducing the wear and tear on mechanical components, thus enhancing the longevity of the system and reducing maintenance requirements. Furthermore, by implementing advanced control strategies, the system ensures that elevators are in operation only when required, preventing idle energy consumption. Over time, this leads to substantial energy savings, contributing to the long-term environmental sustainability of the building. This approach aligns with global efforts to reduce energy usage in urban environments, where buildings are among the largest consumers of energy. In addition to the environmental benefits, the system's efficiency can also play a crucial role in achieving green building certifications, promoting a more sustainable and energy-efficient approach to urban development.

6 Reflection on Individual and Team work (PO(i))

6.1 Teamwork and individual contribution

Name	ID	Contribution
Alauddin Ahmed	2106034	Developed XDC file and directional control
Rakibul Islam Emon	2106040	Optimized state allocation and logic for cost calculation.
Wassie Haque	2106041	Implemented algorithm for efficient lift scheduling(look algorithm), load balancing.
Sanjay Kumar Sen	2106054	Designed and implemented group controller top module logic to manage multiple floor requests

6.2 Log Book of Project Implementation

Date	Milestone achieved	Individual Role
20-12-25	Requirements & Solution Planning	Everyone proposing idea
3-01-26	Coding phase started	Everyone shared future plan
14-01-26	Coding Halfway	Wassie & Alauddin started coding
21-01-26	Coding completed	Everyone goes through the code
24-01-26	Lab check and 1st failure	Sanjay & Emon tested
27-01-26	Recheck initial idea and changes made	Everyone contributed
2-02-26	Finalising idea and re organise circuit	Wassie & Sanjay made some changes
5-02-26	Rechecking code	Emon & Alauddin
22-02-26	3rd failure	Alauddin & Sanjay tested
24-02-26	Update about the project	Everyone gave update and currently facing what problems
02-03-26	Solved lift problem for single lift	Emon & Sanjay tested on FPGA board
05-03-26	Further idea refining	Wassie & Alauddin took that burden
07-03-26	Managed to control 2 lift simultaneously	Everyone helped on debugging and testing
08-04-26	Final test and fine tuning	Everyone was present and organised and debugged

7. Communication to External Stakeholders

7.1 Executive Summary

This project presents the design and implementation of a two-elevator, four-floor group control system, developed using Verilog Hardware Description Language (HDL). The main objective of the system is to optimize the operation of multiple elevators within a building by minimizing waiting time, balancing the load between elevators, and improving service reliability. The system incorporates a centralized group dispatcher that assigns floor requests to elevators based on a cost-optimization strategy, considering factors like distance, direction of movement, and elevator load.

The design employs finite state machines (FSMs) for each elevator car to ensure smooth operation, while a centralized dispatcher determines which elevator should handle each request. A LOOK-based scheduling algorithm is used for efficient directional movement within each elevator, while the dispatcher evaluates requests dynamically, making real-time decisions based on the elevator's status. This approach enables the system to handle simultaneous requests, emergency situations, and full-load conditions effectively.

The system is implemented using a single clock domain to avoid synchronization issues and ensure high efficiency. The project includes a comprehensive testbench that simulates various real-world scenarios to validate the system's functionality. Additionally, a multiplexed 7-segment display interface is integrated to visualize the elevator positions in real-time. This project demonstrates a scalable and modular hardware solution that can be extended to accommodate more floors or additional elevators, with potential applications in smart buildings and automated vertical transportation systems.

7.1 GitHub Link

For access to the full source code, simulation files, and further details about the project implementation, visit our GitHub repository.

GitHub Link : https://github.com/major-45/verilog-FPGA

8. Future work

The current elevator group controller system demonstrates efficient operation for a 2-car, 5-floor building. However, several improvements can be implemented to enhance performance, scalability, and real-world applicability.

i. **Multi-Elevator Expansion:**
The system can be extended to support **more than two elevators**, enabling its use in high-rise buildings. This would require more advanced dispatching strategies to maintain efficiency and avoid congestion.

ii. **Priority-Based Request Handling:**
Future designs may include **priority scheduling**, where certain requests (e.g., emergency, VIP, or service mode) are handled with higher importance than normal calls.

iii. **Fault Detection and Safety Enhancements:**
Additional safety features like **fault detection**, **overload handling**, and **door status monitoring** can be incorporated to make the system more reliable and suitable for real-world deployment.

iv. **IoT and Remote Monitoring:**
The system can be integrated with **IoT platforms** to enable remote monitoring, data logging, and predictive maintenance through cloud-based systems.

v. **Improved User Interface:**
Instead of basic switches and 7-segment displays, a more advanced interface such as **LCD screens or mobile app control** can be introduced for better user interaction.

8. References

i. Jillsmon Varghese, Linju Achu Philip, Shyni Susan Mammen, Yasha Miriam Thomas, and Riboy Cheriyan, "FPGA Implementation of Elevator Controller," *International Journal of Computer Applications*, vol. 97, no. 4, pp. 5-12, 2015.

ii. Mohd Sabir Siddiqui and Kamil Hasan, "Synthesis & Simulation Model of Parallel Lift Controller Using Verilog," *International Journal of Engineering Development and Research*, vol. 3, no. 3, pp. 1-7, 2015.

iii. ChangBum Kim, Kyoung A. Seong, Hyung Lee-Kwang, and Jeong O. Kim, "Design and Implementation of a Fuzzy Elevator Group Control System," *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, vol. 28, no. 3, pp. 277-286, May 1998.

iv. Mohammad Hanif and Nur Mohammad, "Metaheuristic Algorithms for Elevator Group Control System: A Holistic Review," *Soft Computing*, vol. 27, pp. 15905-15936, 2023.